EECS 481: Software Engineering

Kumaran Nathan & Mira Panjaitan

HW6B Report

December 2024

kumarann@umich.edu mnauli@umich.edu

Selected Project

For our project, we chose Aam Digital, an open-source case management software tailored for social workers and NGOs working with vulnerable children and communities. The project is accessible at www.aam-digital.com. The platform aims to streamline data collection and management, enabling organizations to track individual cases, monitor interventions, and measure outcomes more effectively. Aam Digital aggregates data securely and efficiently, allowing for informed decision-making and improved service delivery. It supports offline data entry, customizable forms, and real-time synchronization, making it adaptable to various field conditions and organizational needs.

We chose Aam Digital because of its unique blend of innovative programming practices and its commitment to social good. By offering an open-source tool that enhances the effectiveness of social workers and NGOs, Aam Digital contributes significantly to the welfare of vulnerable populations, such as student support programs, gender equity, marginalized communities, and more. Moreover, the open-source nature of the project ensures that others can build upon and improve the software, fostering collaboration for a better future. Finally, a key factor in our choice was the transparency in the contribution process, as the developers have clearly outlined the steps for making meaningful contributions to the project.

Social Good Indication

We found that Aam Digital qualifies as a Social Good since it aligns with numerous of the seventeen Sustainable Development Goals (SDGs), particularly goals one, three, four, and ten.

I. Goal One: No Poverty
    A. By supporting social workers and NGOs in managing cases, Aam Digital is able to deliver services to front line workers addressing issues affecting vulnerable populations, such as those experiencing poverty.
II. Goal Three: Good Health and Well Being
    A. Aam Digitals platform aids in better tracking and intervention management for individuals and communities, thereby aiding in access to healthcare.
III. Goal Four: Quality Education
    A. Aam Digital supports programs for vulnerable children specifically, helping track and enhance education outcomes for marginalized groups
IV. Goal Ten: Reducing Inequalities
    A. By helping NGOs and social workers to deliver services, the project contributes to reducing inequalities faced by marginalized communities.

Finally, Aam Digital's open-source nature fosters a collaborative and inclusive environment that encourages innovation and shared problem-solving. By allowing developers and organizations

worldwide to access, modify, and improve the software, it creates a dynamic ecosystem where new features and solutions can be continuously developed to address the evolving challenges faced by marginalized communities and individuals. This open approach ensures that the platform remains adaptable and relevant across diverse contexts, promoting long-term sustainability and amplifying its positive impact on those who need it most.

Project Context

Aam Digital is an open-source project designed to support non-governmental organizations and grassroots initiatives by providing a robust case-management system that can be extrapolated to various use cases for the social sector. The developers motivation regarding the project stem from a desire to democratize access to technology for NGOs, enabling them to focus resources on delivering better outcomes for the communities they serve. As the developers have said, NGOs often lack the budget and technical expertise to adopt traditional commercial solutions. Aam Digital is able to bridge the gap, by offering a free, community driven platform that organizations can modify to meet their specific needs.

Aam Digitials significance lies in its ability to transform how NGOs operate, enabling more efficient data management, enhanced transparency, and better reporting. This ensures that resources are directed where they are needed most, and when they are needed the most. Moreover, Aam Digitils open source nature abstracts away the dependency of NGOs on financial contributors, and instead fosters a global community of developers and contributors, who are able to drive innovation and collaboration across NGOs.

Project Governance

Aam Digital uses a combination of tools and processes to facilitate communication and coordination among contributors, striking a balance between formal and informal practices. These mechanisms ensure that volunteers and core team members can collaborate effectively while maintaining flexibility for contributors at various levels of expertise.

Aam Digital begins its coordination efforts on GitHub, where information is provided to streamline the onboarding process for new contributors. The project repository contains detailed documentation, including a tutorial for setup and an overview of the technologies used, making it highly beginner-friendly. Contributors are encouraged to ask questions via GitHub issues or email.

After initial contact, contributors are invited to join the Aam Digital Slack workspace, which serves as the primary channel for real-time communication. Slack facilitates direct interaction with core team members and other contributors, enabling prompt responses to inquiries and fostering a collaborative environment.

Task management is handled using GitHub Projects' kanban-style boards. The board provides a clear overview of all ongoing and pending tasks, categorized by type and priority. Issues labeled as "Community Help Wanted" are specifically targeted at new contributors, often focusing on tasks that do not require extensive familiarity with the codebase. To maintain consistency and clarity, a label definition guide is available to explain the various labels applied to issues. This guide helps contributors understand the status, type, and priority of tasks at a glance.

The process for contributing involves the following steps:

1. Issue Selection: Contributors assign themselves to tasks via the project board, ensuring no duplication of efforts. Only tasks labeled as "In Progress" are actively worked on, while issues in the "Triage / Analysis" column are reserved for further clarification by the core team.
2. Development: Contributors work on their assigned tasks, utilizing the provided documentation and community support.
3. Pull Requests: Upon completion, contributors create a pull request (PR) on GitHub for their work. The core team reviews these PRs and provides feedback as necessary.

Aam Digital's processes for design and quality assurance (QA) are informal. While contributors are encouraged to adhere to standard software engineering practices, no strict guidelines or formal processes are imposed. Feedback on design and QA is provided during pull request reviews, which ensures a level of oversight without imposing rigid requirements.

Task Descriptions

Task 1: Enhancing Entity Imports

This task was focused on updating the `importMapFunction()` in the `EntityDataType` class to address issues caused by type mismatches during data imports. Specifically, importing numeric values were failing to match their string equivalents in the database, which resulted in lookup failures in the management system. Our proposed solution,
The implementation was as follows:

1. **Improving Type Handling & Value Normalization**
    a. The `importMapFunction()` was refactored to include a `normalizeValue()` helper function method which standardized values for

comparison by converting them to strings or numbers respectively. This was able to ensure type consistency during comparisons, allowing for successful map

2. **Robust Matching Logic**
    a. The updated function now retrieves all entities of the specified type, and compares the normalized imported value against the normalized value of the specified field in each entity. If/when a match is found, a corresponding entity ID is returned, otherwise, we return undefined, helping to trace type errors in the future if needed.
3. **Enhances Error Handling**
    a. Error handling was added to manage exceptions, such as issues in entity retrieval or invalid configs.

Task 2: Improving Demo Setup & Data Generation

This task aimed to enhance the original demo setup and data generation for the Aam Digital platform. Initially, the platform was limited to a single configuration file, despite supporting multiple demo sites with use cases such as education, health, and vocational training. As a result, other demo systems operated in a "synced" mode that lacked the ability to auto-generate scenario-specific data dynamically. This limitation made it difficult to customize demo data for different use cases, such as tailoring notes to education or health topics. To address this, the following steps were implemented:

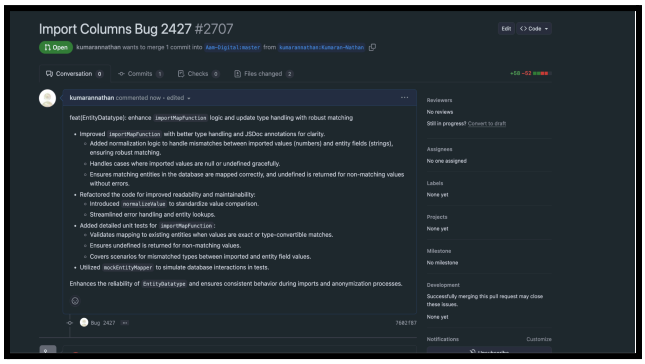
1. **Demo Setup Logic:**
    a. We identified the demo setup logic within the `DemoDataInitializerService` and updated it to detect and read a `demo_entities.json` file from the assets folder.
2. **JSON Parsing & Database Insertion**
    a. We wrote logic to parse the JSON file, validate its structure, and insert the specified documents into the database after the demo data generators had run. This ensured that the data aligned with the expected schema and avoided errors during insertion.
3. **Scenario-Specific Adjustments**
    a. The `demo_entities.json` file was restructured to act as a flexible template for different scenarios, enabling fields and entities to vary based on specific demo use cases (e.g. education-specific notes vs. health-specific notes). This involved reconciling required fields from the `config-fix.ts` file and adjusting dynamically generated fields.

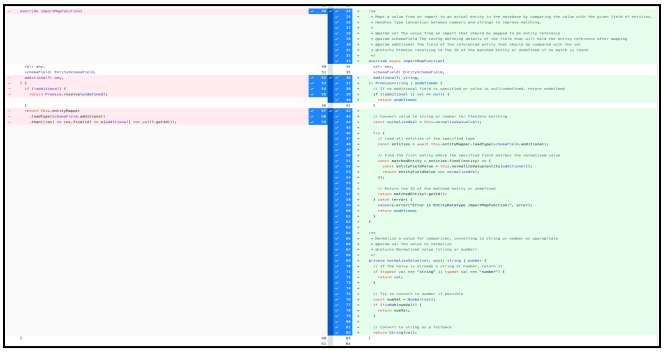
While the original plan also included resolving Keycloak login compatibility issues for iframes, I ultimately focused solely on implementing the `demo_entities.json` functionality. The task's complexity arose from `$localize` being incompatible with static JSON, requiring hardcoded

data instead. Reconciling dynamic placeholders like `PLACEHOLDERS.NOW` with static JSON demanded substituting static values. Understanding the codebase to determine required entities and fields, amid ambiguous documentation, added further challenges. Therefore, I chose to focus on the primary task and achieving the primary goal functionality.

Submitted Artifacts

Task One: Import Columns



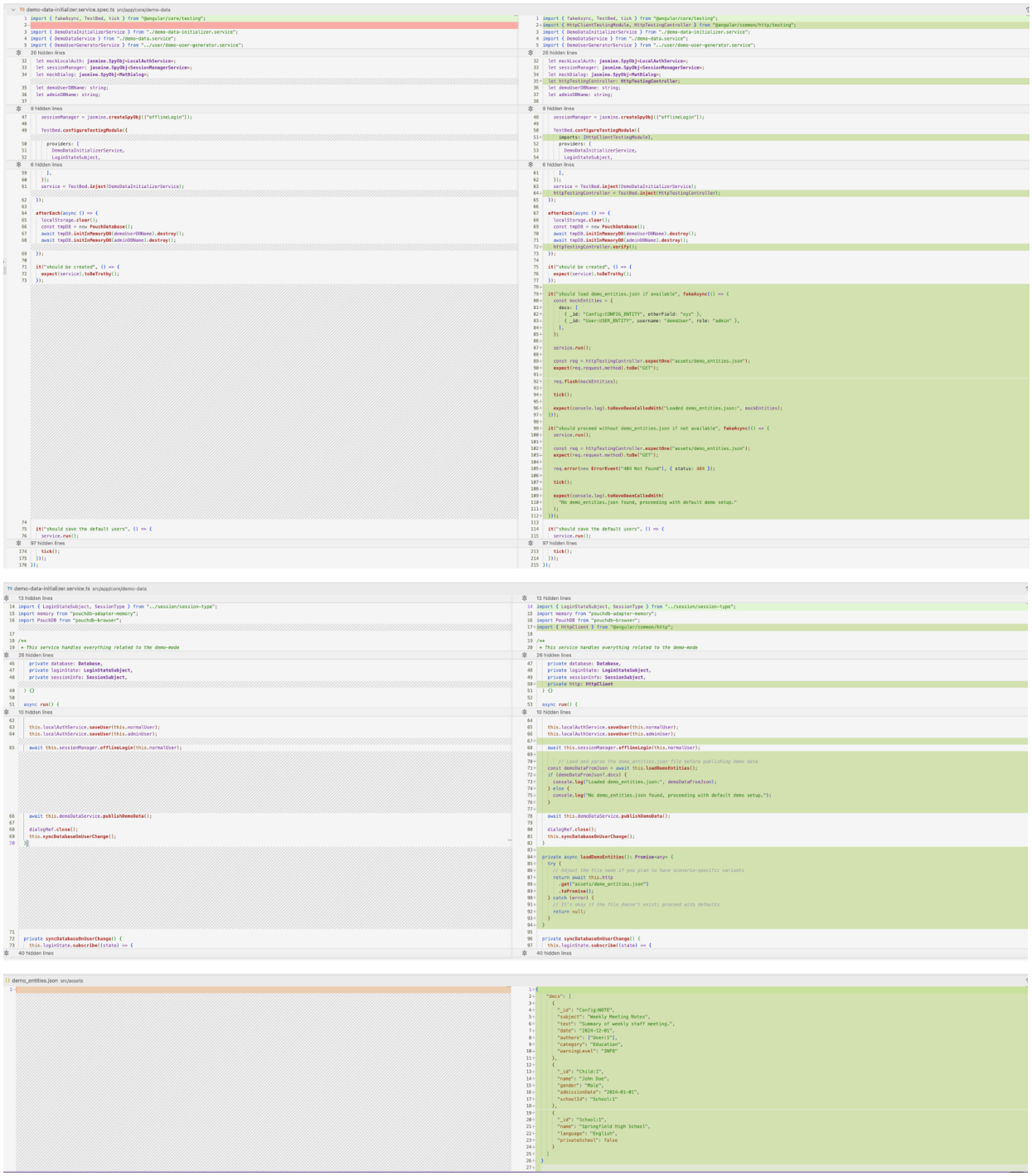


The pull request can be found **here.** This pull request had two main components:

I. Improved type handling for robust entity matching.
II. Added normalization logic to handle type mismatches between numbers and strings.
III. Introduced normalizeValue to standardize value comparison.
IV. Streamlined error handling and entity lookups.

For this bug, in order to tackle the updating of the `importMapFunction()`, work also had to be done in the corresponding class EntityDataType found in `entitydatatype.ts` . In doing so I was able to abstract away components of the function into the class, as well as introduce new functions for ensuring type correctness within the file `importMapFunction()` was present in.

Task Two: Import Improving Demo Setup & Data Generation



The pull request can be found here. The key components are as follows:

I. Integration of `demo_entities.json` File: Implemented logic in DemoDataInitializerService to detect, load, and parse the `demo_entities.json` file from the assets folder

II. Support for Scenario-Specific Data: The addition of `demo_entities.json` allows for customizable demo data tailored to different use cases.
III. Sample File for Testing: Added a sample `demo_entities.json` file to demonstrate the structure and to aid in testing and future contributions.

For this task, implementing the logic to read and parse demo_entities.json required adjustments in `DemoDataInitializerService`, ensuring compatibility with existing processes and maintaining backward compatibility with `config-fix.ts`. This initial implementation lays the groundwork for future enhancements, such as dynamically configuring additional demo parameters for more flexible and robust data generation.

Quality Assurance Strategy

Throughout the project, our quality assurance practices followed two core principles: **local testing** and **pair programming,** and **code review** with Aam Digital developers.

Local testing was tailored to the requirements of each task, adapting to the specific technical specifications involved. At a high level, this process included running the Aam Digital project on our local systems and leveraging unit tests to validate that the behavior aligned with expectations. By testing locally, we were able to identify and resolve issues efficiently before they impacted the main project. This approach reduced technical overhead and ensured smoother integration. To simulate interactions between components, we incorporated mock functions and designed tests to cover a range of edge cases, enhancing the robustness of our implementation.

Pair programming was another key practice in our workflow. We collaborated closely, with one partner focusing on writing code and the other providing real-time review and feedback. This dynamic approach fostered collaboration and significantly reduced the need for formal code reviews within our team. By continuously checking and refining the code as it was written, we ensured higher quality and alignment with project requirements while maintaining a shared understanding of the task at hand.

Code reviews with the development team were conducted via Slack and GitHub discussions. Once tasks were implemented and tested locally, we submitted pull requests for review. During this process, developers provided feedback on the code's functionality, adherence to project guidelines, and maintainability. These conversations often sparked valuable insights, leading to further refinement of our work. By leveraging Slack for quick clarifications and GitHub for more detailed discussions, we maintained an efficient and collaborative review process that ensured the quality and integration-readiness of our contributions.

Quality Assurance Evidence

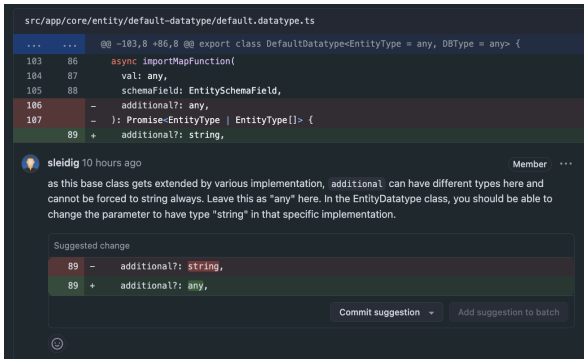
```
import { TestBed } from '@angular/core/testing';
import { EntityDatatype } from './entity.datatype';
import { EntityMapperService } from '../../entity/entity-mapper/entity-mapper.service';
import { EntitySchemaField } from '../../entity/schema/entity-schema-field';

describe('EntityDatatype', () => {
  let entityDatatype: EntityDatatype;
  let mockEntityMapper: jasmine.SpyObj<EntityMapperService>;

  beforeEach(() => {
    // Create a mock EntityMapperService
    mockEntityMapper = jasmine.createSpyObj('EntityMapperService', ['loadType']);

    TestBed.configureTestingModule({
      providers: [
        EntityDatatype,
        { provide: EntityMapperService, useValue: mockEntityMapper }
      ]
    });

    entityDatatype = TestBed.inject(EntityDatatype);
  });

  describe('importMapFunction', () => {
    const mockSchemaField: EntitySchemaField = {
      additional: 'MockEntity',
      isArray: false,
    };

    it('should return/map to existing entity in DB if imported value matches the given field', async () => {
      // Arrange
      const mockEntities = [
        {
          getId: () => 'entity1',
          name: 'Test Entity'
        }
      ];
      mockEntityMapper.loadType.and.returnValue(Promise.resolve(mockEntities));

      // Act
      const result = await entityDatatype.importMapFunction('Test Entity', mockSchemaField, 'name');

      // Assert
      expect(result).toBe('entity1');
      expect(mockEntityMapper.loadType).toHaveBeenCalledWith('MockEntity');
    });

    it('should handle number to string matching for imported value', async () => {
      // Arrange
      const mockEntities = [
        {
          getId: () => 'entity1',
          code: '123'
        },
        {
          getId: () => 'entity2',
          code: 456
        }
      ];
      mockEntityMapper.loadType.and.returnValue(Promise.resolve(mockEntities));
```

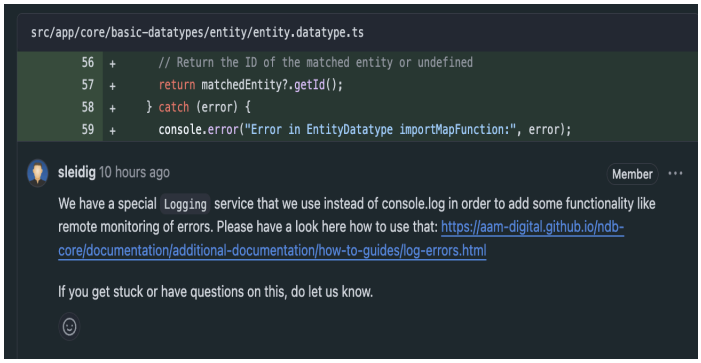
```
      // Act
      const result = await entityDatatype.importMapFunction('Non Existent', mockSchemaField, 'name');

      // Assert
      expect(result).toBeUndefined();
    });

    it('should return undefined if no additional field is specified', async () => {
      // Act
      const result = await entityDatatype.importMapFunction('Test', mockSchemaField, undefined);

      // Assert
      expect(result).toBeUndefined();
      expect(mockEntityMapper.loadType).not.toHaveBeenCalled();
    });

    it('should return undefined if imported value is null or undefined', async () => {
      // Act
      const resultNull = await entityDatatype.importMapFunction(null, mockSchemaField, 'name');
      const resultUndefined = await entityDatatype.importMapFunction(undefined, mockSchemaField, 'name');

      // Assert
      expect(resultNull).toBeUndefined();
      expect(resultUndefined).toBeUndefined();
      expect(mockEntityMapper.loadType).not.toHaveBeenCalled();
    });
  });
});

function beforeEach(arg0: () => void) {
  throw new Error('Function not implemented.');
}


function expect(result: any) {
  throw new Error('Function not implemented.');
}
```

Above are **unit tests** implemented along with the `MockEntityMapper()` to test entity mapping from the EntityDataType Class, as well as the normalization of values.

Above are **unit tests** in `demo-data-initializer.service.spec.ts` to validate the correct loading and parsing of `demo_entities.json` and ensure fallback to `config-fix.ts` when the file is unavailable.

Below are examples of **code review** with the development team at Aam Digital .





Plan Updates

Task 1 Plan Updates:
As we progressed through the project & Task 1, we found that we had to diverge from our original plan/scope. While we had originally planned to utilize Jasmine Unit Testing in order to test the `importMapFunction()` and `EntityDataType` class, this proved much harder to learn than we previously anticipated in light of a large, complex codebase that took much longer to understand than we had expected.

Jasmine Unit Testing
Although our original plan was to write Jasmine Unit tests for the `importMapFunction()`, which would validate the handling of type mismatches, after reviewing much documentation and the codebase, we found that local testing would prove more time efficient. The complexity of learning, integrating, and testing with Jasmine unit tests correctly within our expected time effort (ETE) would not have made sense within the time framework we had set. Consequently, we decided to focus on using mock functions and data instead, testing core functionality that would affect the bug listed.

Activity	Planned Time Effort	Actual Time Effort	Notes
Initial Code Review	3 Hours	3 Hours	Completed.

Adding Type and jsDocs	5-7 Hours	~5.5 Hours	Completed.
Unit Testing	4-5 Hours	8+ Hours	Abandoned[1]
Logic Update TH	5 Hours	6 Hours	Over ETE[2]
QA/Testing/Review	3 Hours	4 Hours	Over ETE[3]

[1] Unit Testing shifted from Jasmine unit testing to local testing
[2] Logic Update Type Handling (TH) went over the ETE due to extra time needed to comprehend codebase documentation for various types.
[3] QA/Testing/Review went marginally over our ETE due to the time handling edge cases.

For the second task, we found that while initially the plan called for separating tasks into distinct phases of exploration and development, in practice, the exploration and development phases merged, as familiarizing myself with the codebase required significant hands-on experimentation. For example, the planned 2-hour exploration phase expanded to approximately 6 hours, as I needed to understand not only the existing demo setup logic but also how dynamic elements were integrated.

Development also extended beyond the planned 9 hours to around 14 hours, as addressing challenges with scenario-specific configurations required iterative testing and adjustments. For instance, creating a template `demo_entities.json` and reconciling hard-coded data with flexible, user-customizable templates added unforeseen complexity. Additionally, challenges like managing localization and handling dynamic vs. static data necessitated rethinking the approach, delaying the schedule.

As a result, tasks like Keycloak Login Compatibility Adjustment were deprioritized to focus on completing the core demo setup task. This decision aligned with the primary goal of supporting configurable demo data but altered the overall scope and timeline. Testing and QA were condensed into troubleshooting during development, rather than occurring as separate phases, as time constraints made this a more practical approach. Ultimately, while deviations from the schedule occurred, the adjustments ensured that the primary task was addressed comprehensively, even if fewer deliverables were achieved overall.

Experiences & Future Recommendations

Working on an open-source project, especially Aam Digitals management system, was a very rewarding, yet also challenging experience. Engaging with a large, pre-existing project codebase introduced complexities we had not yet encountered or anticipated, particularly with

understanding a new organization's architecture, syntax, and structure. Aam Digital was able to streamline this process, by including a clear contribution guide as well as quick, concise communication within the Slack. The codebase was very well maintained, however given the vast amount of different contributors, there is sometimes a lack of consistency with clarity, and the provided information to tackle issues, which resulted in a large time investment in understanding the code base. The extensive codebase, coupled with their use of technologies such as Angular and TypeScript of which we had an intermediate level of understanding, resulted in us underestimating the effort required to familiarize ourselves with the intricacies of the codebase.

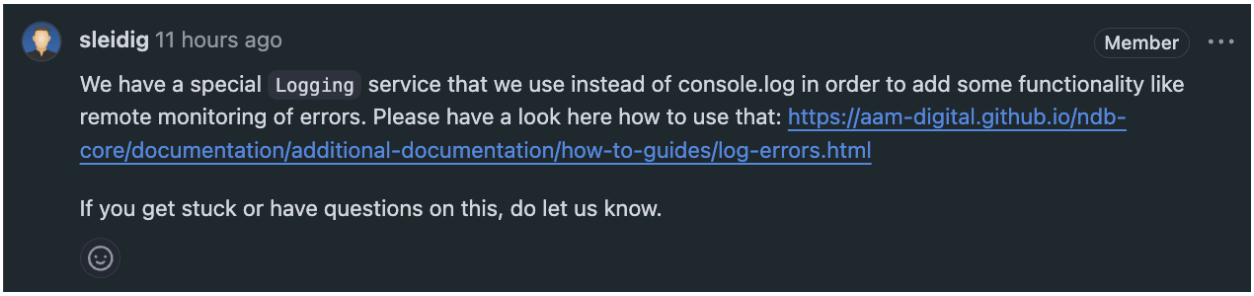
Challenges Faced

One of the main challenges we faced was understanding and modifying a pre-existing codebase without causing structural changes to other related components. The `importMapFunction` task required us to delve deep into the codebase, find the `EntityDataType` class, which had its own set of complex data dependencies and relations to other components throughout the codebase. While the developers had organized issues effectively, understanding the underlying logic and implementing robust solutions within the time constraints proved more difficult than we had initially expected. For task one, the biggest hurdle was understanding, and trying to implement Jasmine unit tests. Despite comprehensive documentation, we found its integration difficult, and that local testing would be more efficient.

For our other task, implementing the ability to read and parse the `demo_entities.json file` required navigating the tightly coupled architecture of the `DemoDataInitializerService` and understanding its dependencies, such as `config-fix.ts`. It involved ensuring backward compatibility with the existing configuration while introducing new functionality that could be customized for different demo scenarios. This added an extra layer of complexity, as it required balancing the need for new functionality with preserving existing behavior.

Another significant challenge was reconciling dynamic and static data handling. For example, pre-existing configurations such as `PLACEHOLDERS.NOW` for dynamic dates were straightforward in TypeScript but did not translate seamlessly into JSON. This required critical thinking to decide whether and how such placeholders could be represented in a static JSON file, or whether alternative approaches were needed. These nuanced decisions underscored the learning curve associated with navigating and adapting to the established practices within Aam Digital's codebase.

Collaboration with the Community



The Aam Digital team and contributors fostered a collaborative and open environment, welcoming to beginners which aided greatly in our process. Their responsiveness both on github and Slack proved immensely useful, as they provided concise explanations, and gave leeway with us being new members of the Aam Digital contributor community. One great instance of feedback was from @sleidig in response to our testing methods, shown below.

This type of feedback - citing our error, referencing external documentation, and offering additional help if needed was particularly helpful. Initially, we tried to ensure everything was correct before making a PR, however, after receiving insightful feedback such as the one above, we felt help was much more accessible than previously thought. However, the reliance on asynchronous communication meant that clarifications sometimes took longer than expected, slowing our progress. For example, while creating a pull request for the `demo_entities.json` functionality, we encountered an unexpected issue where we were unable to submit the PR because we were not designated as collaborators on the project. This required one of us to submit a pull request on behalf of the other, adding an extra layer of coordination and delay. This experience highlighted a potential area for improvement in onboarding new contributors by providing clearer guidance on the prerequisites for contributing, such as setting up permissions or understanding the project's workflow. This information, while essential, was not explicitly stated in the project's documentation, which could streamline the process for future contributors.

What Worked, What Didn't

One aspect of our approach to the project that worked was breaking the tasks into smaller steps. This approach adhered to the **agile development** principles, where we took an iterative approach. This iterative approach meant we could tackle smaller, well-defined goals in each step, which made the larger task less overwhelming. Frequent check-ins and adjustments ensured that we could adapt to unexpected challenges, such as understanding the nuances of the codebase or addressing unforeseen bugs, without derailing our overall timeline. This methodology also ensured continuous progress and kept the project moving forward, even when obstacles arose. By following the agile development process, we were able to focus on understanding the existing

functionality first in order to **create a plan**[1], followed by **designing**[2] how we would approach implementing the code. Next we were able to proficiently **develop**[3] as we had gained a solid understanding of the codebase, as well as a robust design framework to follow as we programmed. Following development, we implemented local **testing**[4] for quality assurance, and to ensure what we had implemented truly functioned as planned. The next step in the process involved **deployment**[5] where we made pull requests, subsequently followed by code **review**[6] with the developers.

While our overall process adhered well to agile development principles, one area that presented significant challenges was understanding and testing within the existing framework. For example, our initial plan included leveraging Jasmine to validate our changes to the `importMapFunction` method. However, the complexity of the pre-existing codebase and our limited experience with Jasmine created unexpected hurdles. Despite reviewing documentation and examples, integrating Jasmine within the context of the project proved more time-consuming than anticipated.

Another major challenge was dealing with dynamic placeholders like `PLACEHOLDERS.NOW` and other values generated by the system, which could not be directly represented in a static JSON file. This limitation required creative workarounds, such as simulating dynamic behavior in tests or documenting how future users might configure such placeholders. This complexity slowed down progress and highlighted areas where additional flexibility in the framework might be beneficial.

Additionally, the time invested in attempting to configure Jasmine could have been better spent exploring alternative testing methods earlier in the process. This delay meant we had to shift our focus to local testing using mock data to validate functionality, which, while effective, did not provide the comprehensive coverage we had initially planned. More time could have also been dedicated towards exploring and mapping the codebase. This would have allowed me to identify dependencies and key relationships earlier, saving time spent debugging and backtracking later. In hindsight, incorporating a dedicated exploratory phase for learning new tools and frameworks at the outset of the project would have mitigated these issues and allowed for a more streamlined testing phase.

Advice For Future Students

Kumaran Nathan: Start early, allocate time for learning unfamiliar tools, and don't hesitate to reach out to open-source communities—they are usually welcoming and helpful.

Mira Panjaitan: Break tasks into smaller steps and continuously check your progress against the requirements; this iterative approach will save you from getting overwhelmed.

We give permission for future students to see our materials.